EcoScore Engine

Plataforma de Incentivo a Ações Sustentáveis

Domain Driven Design Using Java — Sprint 4
Documentação final da API RESTful (Spring Boot)

Campo	Informação
Nome da equipe	GaiaTech
Nome da solução	EcoScore Engine — API RESTful
Empresa parceira	SoulUp (Challenge FIAP 2026)
Disciplina	Domain Driven Design Using Java
Turma	1TDSPV
Tecnologias	Java 21 · Spring Boot 4.1 · JDBC (DAO / Connection Factory) · ModelMapper · Oracle · Docker · Render
API publicada	https://ecoscoreengine-api.onrender.com
Repositório GitHub	https://github.com/Leonardo-2112/EcoScoreEngine-API

Integrantes

Nome completo	RM	Turma
Leonardo Afonço Sousa	RM573982	1TDSPV
Matheus Jones Sousa de Freitas	RM573559	1TDSPV
Syang Cristina Gomes Souza	RM569259	1TDSPV
Joao Machado Nascimento	RM572364	1TDSPV
Vitor Gonçalves de Souza	RM569023	1TDSPV

Sumário

1. Objetivo e escopo do projeto
2. Funcionalidades implementadas
3. Arquitetura, camadas e padrões de projeto
4. Tabela de endpoints (API RESTful)
5. Protótipo — telas implementadas
6. Modelo Entidade-Relacionamento (MER)
7. Diagrama de classes atualizado
8. Procedimentos para rodar a aplicação
9. Deploy na nuvem
10. Testes realizados

1. Objetivo e escopo do projeto

O **EcoScore Engine** transforma ações sustentáveis do cidadão em **pontos, missões, níveis e créditos**, que podem ser convertidos e sacados em reais. Empresas parceiras patrocinam **campanhas ESG** e acompanham o impacto por indicadores agregados. Fluxo da solução: *Ação → Pontos → Missões → Nível → Bônus → Créditos → Saque (R$)*.

Nesta sprint, o projeto Java deixou de ser uma aplicação de console e passou a ser uma **API RESTful em Spring Boot**. A API é consumida pelo Front-end em React (Vercel) e grava os dados no Oracle modelado em Building Relational Database.

Dentro do escopo:

- cadastro e login dos três perfis (participante, validador e empresa), com bloqueio da conta após 5 senhas erradas;
- registro de ações com cálculo de pontos, limite diário e fila de validação;
- missões, níveis, desafios temáticos e ranking;
- carteira, com conversão de pontos em créditos e saque;
- módulo B2B com empresas, campanhas, métricas e dashboard;
- catálogo de ações, categorias e desafios.

Fora do escopo (limites):

- o pagamento PIX é **simulado** (o saque é gravado como processado);
- a evidência é descrita em texto (sem upload de arquivo);
- a identificação do usuário logado é feita pelo header `X-Usuario-Id`, sem token JWT;
- o bônus de campanha fica fora da fórmula de pontos (regra dos slides da mentoria).

2. Funcionalidades implementadas

Funcionalidade	Descrição	Endpoints
Cadastro e login	Cadastro ampliado com validação de CPF (dígito verificador), idade mínima de 16 anos e UF. Senha guardada como hash BCrypt. Login com bloqueio após 5 tentativas erradas (RN02) e desbloqueio pelo validador.	`/auth/cadastro`, `/auth/login`, `/usuarios/{id}/desbloqueio`
Perfil do usuário	Consulta, edição (o CPF não pode mudar) e exclusão lógica (status INATIVO).	`/usuarios/{id}`
Registro de ação	Pontos = TRUNC((impacto × 10 × 1/repetições no dia × confiabilidade) × bônus do nível). No máximo 2 registros por dia da mesma ação. Ação comprovada é creditada na hora; parcial ou sem comprovação fica PENDENTE. O registro é vinculado à campanha ESG vigente.	`/registros`
Validação (validador)	Fila de pendentes. O validador aprova (credita os pontos, podendo ajustar a confiabilidade) ou nega (0 pontos, com justificativa obrigatória).	`/registros?status=PENDENTE`, `/registros/{id}/validacao`
Missões, níveis e desafios	A cada 3 ações creditadas, 1 missão concluída. Os níveis vão por missões (0–4, 5–9, 10–19, 20+), com bônus ×1,0 a ×1,3, e o nível nunca diminui. Os desafios temáticos têm progresso próprio.	`/usuarios/{id}/painel`, `/usuarios/{id}/desafios`
Carteira	Conversão de 100 pontos em 1 crédito (irreversível). Saque com mínimo de 10 créditos (10 créditos = R$ 1,00). As duas operações usam transação e SELECT FOR UPDATE.	`/usuarios/{id}/carteira`, `/conversoes`, `/saques`
Ranking	Participantes ativos ordenados pelos pontos acumulados; converter pontos não reduz a posição.	`/ranking`
Módulo B2B	Cadastro de empresa com CNPJ validado. A empresa nasce PENDENTE e o validador a ativa (RN03). Campanhas ESG passam por aprovação (RN16). Métricas e Dashboard ESG mostram apenas dados agregados (RN17).	`/empresas`, `/campanhas`, `/dashboard/esg`
Catálogo	CRUD de ações, categorias e desafios (escrita pelo validador). As exclusões são lógicas para preservar o histórico.	`/acoes`, `/categorias`, `/missoes`, `/niveis`
Diagnóstico	Página inicial e verificação de saúde da API e do banco.	`/`, `/health`

3. Arquitetura, camadas e padrões de projeto

Camada / pacote	Responsabilidade	Classes
model	Entidades do domínio, uma para cada tabela do MER, com encapsulamento, construtores, getters/setters e métodos de domínio. **As associações são feitas por referência a objetos.**	15 classes + 9 enums
dao (padrão DAO)	Classes `@Repository` (como na aula): SQL em constantes, método privado `getXxx(ResultSet)` que monta o objeto, PreparedStatement com try-with-resources e `EntidadeNaoEncontradaException` quando o registro não existe. Os objetos associados são montados por JOIN. Os métodos usados em transação recebem a Connection do BO.	14 DAOs
factory (Connection Factory)	`ConnectionFactory.getConnection()` entrega a conexão com o Oracle a partir do DataSource do Spring. As credenciais vêm de variáveis de ambiente (DB_URL, DB_USER, DB_PASSWORD).	ConnectionFactory
bo (regras de negócio)	Classes `@Service` com as validações, as regras de gamificação e o controle de acesso por perfil. As operações que gravam em várias tabelas usam transação explícita: `setAutoCommit(false)`, `commit()` e `rollback()` no catch.	13 BOs + ValidadorDados
resource (REST)	Classes `@RestController`: recursos no plural, verbos HTTP, `ResponseEntity` com o status correto e `UriComponentsBuilder` para o header Location no 201. O **ModelMapper** converte DTO ↔ Model.	10 Resources
dto	Classes `XxxRequestDto` (entrada, com Bean Validation) e `XxxResponseDto` (saída, sem senha e sem dados internos).	13 RequestDto + 23 ResponseDto
config	**CorsFilter**: libera o Front (Vercel) a consumir a API e responde ao pre-flight OPTIONS. **ModelMapperConfig**: disponibiliza o ModelMapper.	CorsFilter, ModelMapperConfig
exception	`EntidadeNaoEncontradaException` (checada, como na aula) e exceções de negócio. O `ApiExceptionHandler` (@RestControllerAdvice) devolve o status correto com um `ErrorResponseDto {erro, detalhe}`.	7 exceções + handler

3.1 Transação nas operações compostas (ex.: conversão de pontos)

```
Connection conexao = connectionFactory.getConnection();
try {
    conexao.setAutoCommit(false);
    Usuario usuario = usuarioDao.buscarPorIdParaAtualizar(conexao, usuarioId);   // SELECT ... FOR UPDATE
    Carteira carteira = carteiraDao.buscarPorUsuarioParaAtualizar(conexao, usuario);
    usuario.debitarSaldo(pontos);           usuarioDao.atualizarPontuacao(conexao, usuario);
    carteira.creditar(creditos, agora);     carteiraDao.atualizar(conexao, carteira);
    conversaoDao.cadastrar(conexao, conversao);
    conexao.commit();
} catch (Exception e) {
    conexao.rollback();
    throw e;
} finally {
    conexao.close();
}
```

3.2 Atendimento ao feedback da Sprint 3

> Feedback do professor: *“Nas classes de modelo, os atributos devem seguir a orientação a objetos, ou seja, deve ser do tipo da referência da outra classe e não somente o id (como FK). Ex.: HistoricoSaque possui um Usuario usuario e não um int idUsuario.”*

Classe	Sprint 3 (id solto)	Sprint 4 (referência a objeto)
HistoricoSaque	`int idUsuario`	`Usuario usuario`
HistoricoConversao	`int idUsuario`	`Usuario usuario`
Carteira	`int idUsuario`	`Usuario usuario`
Usuario	`int idNivel`	`Nivel nivel; EmpresaParceira empresa`
Acao	`int idCategoria`	`CategoriaAcao categoria`
RegistroAcao	`Integer idCampanha`	`CampanhaEsg campanha; Usuario validador`
CampanhaEsg (nova)	–	`EmpresaParceira empresa; List<CampanhaAcao> acoes`
CampanhaAcao / MissaoAcao (novas)	–	`CampanhaEsg campanha / Missao missao; Acao acao`
UsuarioMissao (nova)	–	`Usuario usuario; Missao missao`
MetricaEsg (nova)	–	`CampanhaEsg campanha`

Outras melhorias em relação à Sprint 3:

- a única conexão estática foi substituída pela ConnectionFactory com pool de conexões, e os statements passaram a ser fechados (try-with-resources);
- as operações compostas usam transação;
- a senha agora é guardada como hash BCrypt;
- as credenciais saíram do código;
- os erros viraram exceções com status HTTP, no lugar dos `println`;
- o ranking passou a ser ordenado e o bônus de nível foi aplicado.

4. Tabela de endpoints (API RESTful)

URL base: https://ecoscoreengine-api.onrender.com. Corpo das requisições e respostas: JSON. Nas rotas protegidas, envie o header `X-Usuario-Id` com o id do usuário logado.

Códigos de status:

- 200 OK · 201 Created · 204 No Content;
- 400 dados inválidos · 401 não identificado · 403 sem permissão · 404 não encontrado · 405 método não permitido;
- 409 conflito ou duplicidade · 422 regra de negócio · 423 conta bloqueada;
- 500 erro interno · 503 banco indisponível.

Verbo	URI	Acesso	Descrição	Status
GET	`/`	público	Informações da API	200
GET	`/health`	público	Saúde da API e do banco	200, 503
POST	`/auth/login`	público	Login (bloqueia após 5 tentativas erradas)	200, 400, 401, 403, 423
POST	`/auth/cadastro`	público	Cadastro de participante	201, 400, 409
GET	`/usuarios`	validador	Lista de usuários (?perfil=&status=)	200, 401, 403
GET	`/usuarios/{id}`	próprio/validador	Dados do usuário	200, 401, 403, 404
PUT	`/usuarios/{id}`	próprio/validador	Edição do perfil	200, 400, 403, 404, 409
DELETE	`/usuarios/{id}`	próprio/validador	Exclusão lógica (status INATIVO)	204, 403, 404, 422
PATCH	`/usuarios/{id}/desbloqueio`	validador	Desbloqueio de conta SUSPENSA	200, 403, 404, 422
GET	`/usuarios/{id}/painel`	próprio/validador	Nível, missão em andamento, carteira, CO_2 e posição no ranking	200, 403, 404, 422
GET	`/usuarios/{id}/desafios`	próprio/validador	Progresso nos desafios temáticos	200, 403, 404
GET	`/usuarios/{id}/registros`	próprio/validador	Histórico de registros (?status=)	200, 403
GET	`/usuarios/{id}/carteira`	próprio/validador	Saldo de créditos e valor em R$	200, 403, 404

POST	`/usuarios/{id}/conversoes`	próprio participante	Converte pontos em créditos (100 pts = 1 crédito)	201, 403, 422
GET	`/usuarios/{id}/conversoes`	próprio/validador	Histórico de conversões	200, 403
POST	`/usuarios/{id}/saques`	próprio participante	Saque (mínimo de 10 créditos; 10 créditos = R$ 1,00)	201, 400, 403, 422
GET	`/usuarios/{id}/saques`	próprio/validador	Histórico de saques	200, 403
POST	`/registros`	participante	Registra uma ação sustentável	201, 400, 403, 404, 422
GET	`/registros`	logado	Lista de registros (?status=PENDENTE&usuarioId=)	200, 401, 403
GET	`/registros/{id}`	dono/validador	Detalhe do registro	200, 403, 404
PATCH	`/registros/{id}/validacao`	validador	Aprova ou nega um registro pendente	200, 400, 403, 404, 422
GET	`/acoes`	público	Catálogo (?status=ATIVA|INATIVA|TODAS&categoriaId=)	200, 400
GET	`/acoes/{id}`	público	Detalhe da ação	200, 404
POST	`/acoes`	validador	Cria uma ação	201, 400, 403, 404, 409
PUT	`/acoes/{id}`	validador	Edita uma ação	200, 400, 403, 404, 409
DELETE	`/acoes/{id}`	validador	Exclusão lógica (status INATIVA)	204, 403, 404, 422
GET	`/categorias` · `/categorias/{id}`	público	Categorias	200, 404
POST	`/categorias`	validador	Cria uma categoria	201, 400, 403, 409
PUT	`/categorias/{id}`	validador	Edita uma categoria	200, 403, 404, 409

DELETE	`/categorias/{id}`	validador	Exclui (somente sem ações vinculadas)	204, 403, 404, 409
GET	`/missoes · /missoes/{id}`	público	Desafios temáticos	200, 404
POST	`/missoes`	validador	Cria um desafio	201, 400, 403, 404, 409
PUT	`/missoes/{id}`	validador	Edita um desafio	200, 403, 404, 409
DELETE	`/missoes/{id}`	validador	Exclusão lógica (status INATIVA)	204, 403, 404, 422
GET	`/niveis`	público	Níveis e bônus	200
GET	`/ranking`	público	Ranking pelos pontos acumulados (?limite=10)	200
POST	`/empresas`	público	Cadastro B2B: empresa PENDENTE e representante	201, 400, 409
GET	`/empresas`	validador	Lista de empresas (?status=PENDENTE)	200, 403
GET	`/empresas/{id}`	dona/validador	Dados da empresa	200, 403, 404
PUT	`/empresas/{id}`	dona/validador	Edição (o CNPJ não pode mudar)	200, 400, 403, 409, 422
PATCH	`/empresas/{id}/status`	validador	Ativação, suspensão ou inativação	200, 400, 403, 404, 422
GET	`/campanhas`	público/empresa/validador	Campanhas (público vê só as ATIVAS)	200
GET	`/campanhas/{id}`	público se ATIVA	Detalhe da campanha	200, 401, 403, 404
POST	`/campanhas`	empresa	Cria campanha (status PENDENTE)	201, 400, 403, 404, 409, 422
PUT	`/campanhas/{id}`	empresa dona	Edita campanha PENDENTE	200, 400, 403, 409, 422

DELETE	`/campanhas/{id}`	dona/validador	Cancelamento (status CANCELADA)	204, 403, 404, 422
PATCH	`/campanhas/{id}/status`	validador	Aprovação, encerramento ou cancelamento	200, 400, 403, 404, 422
GET	`/campanhas/{id}/metricas`	dona/validador	Métricas ESG agregadas	200, 403, 404
GET	`/dashboard/esg`	empresa/validador	Indicadores agregados das campanhas	200, 401, 403

4.1 Exemplos de requisição e resposta

```
POST /registros
X-Usuario-Id: 1
Content-Type: application/json

{ "acaoId": 4, "tipoEvidencia": "COMPROVADA", "evidencia": "Foto da muda plantada" }

201 Created   Location: /registros/405
{ "id": 405, "acao": { "id": 4, "nome": "Plantio de Arvore", "impacto": 5, "pontosBase": 50 },
  "repeticoes": 1, "confiabilidade": 1.0, "pontosGerados": 65, "validacao": "VALIDADA",
  "observacao": "(5 x 10 x 1/1 x 1.0) x 1.3 = 65", ... }
```

```
POST /registros   (3º registro da mesma ação no dia)

422 Unprocessable Content
{ "erro": "Regra de negocio",
  "detalhe": "Limite diario atingido: no maximo 2 registros por dia da acao 'Plantio de Arvore' (RN14)." }
```

```
POST /auth/cadastro   (campos invalidos)

400 Bad Request
{ "email": "E-mail invalido.", "cpf": "CPF deve ter 11 digitos." }
```

5. Protótipo — telas implementadas

Capturas reais do Front-end (React + Vite + TypeScript), publicado na Vercel e consumindo esta API.
Endereço do Front: **a preencher após o deploy do Front-end**.

PENDENTE
NN_Nome_da_Tela.png

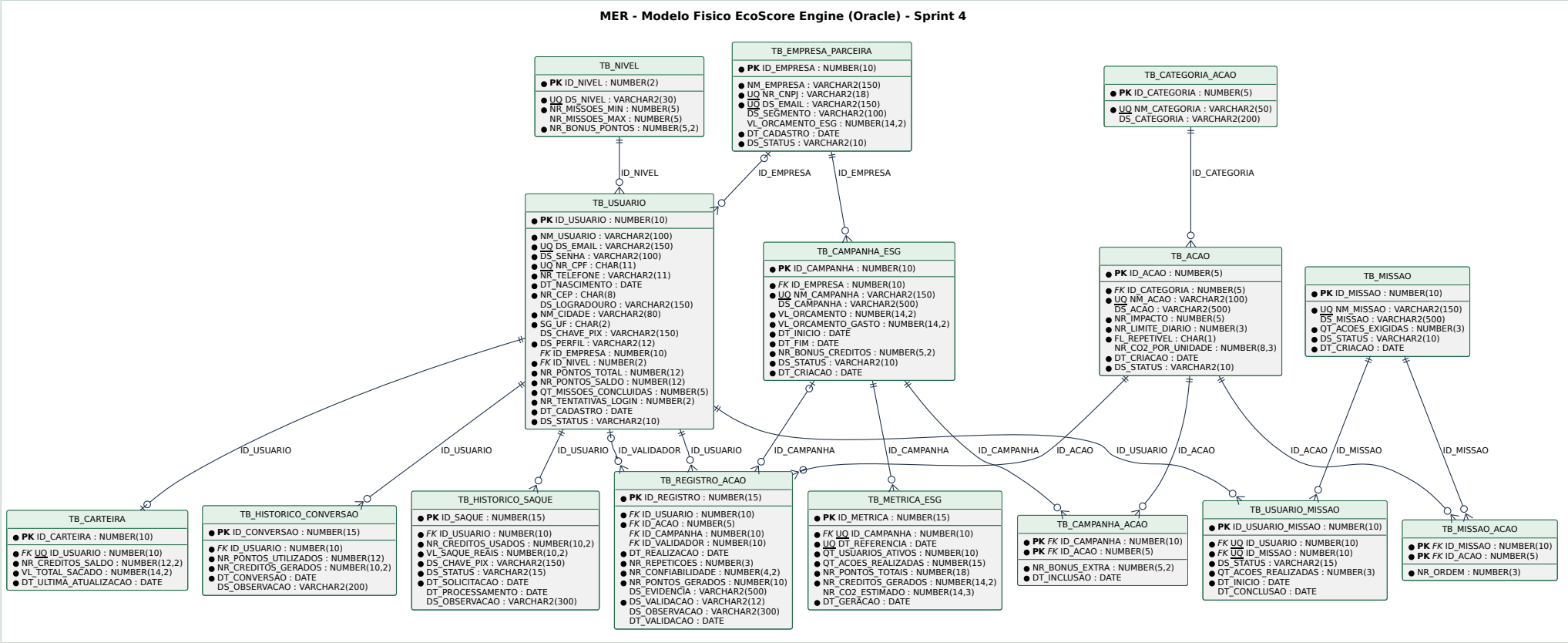
As telas do Front-end serão incluídas após a conclusão do Plano 3 (Front-end).

6. Modelo Entidade-Relacionamento (MER)

Modelo físico no Oracle, gerado a partir do script `00_EcoScore_DDL.sql` da disciplina de Building Relational Database. São 15 tabelas, 15 chaves primárias, 18 chaves estrangeiras e as restrições UNIQUE e CHECK das regras de negócio. Notação: **PK** chave primária, *FK* chave estrangeira, UQ única, ● obrigatório.

Cardinalidades:

- **1:1** — Usuário e Carteira;
- **1:N** — Nível e Usuário; Categoria e Ação; Empresa e Campanha; Usuário e Registros/Histórico;
- **N:N** resolvidas por tabelas associativas — Campanha × Ação (TB_CAMPANHA_ACAO); Missão × Ação (TB_MISSAO_ACAO); Usuário × Missão (TB_USUARIO_MISSAO).



MER — modelo físico do EcoScore Engine (Sprint 4).

7. Diagrama de classes atualizado

Diagramas gerados a partir do código-fonte entregue, o que garante que sejam compatíveis com o projeto: atributos com tipos e visibilidade, métodos de domínio, associações com multiplicidade e composição (◆). Os atributos privados (−) são acessados por getters/setters públicos (+).

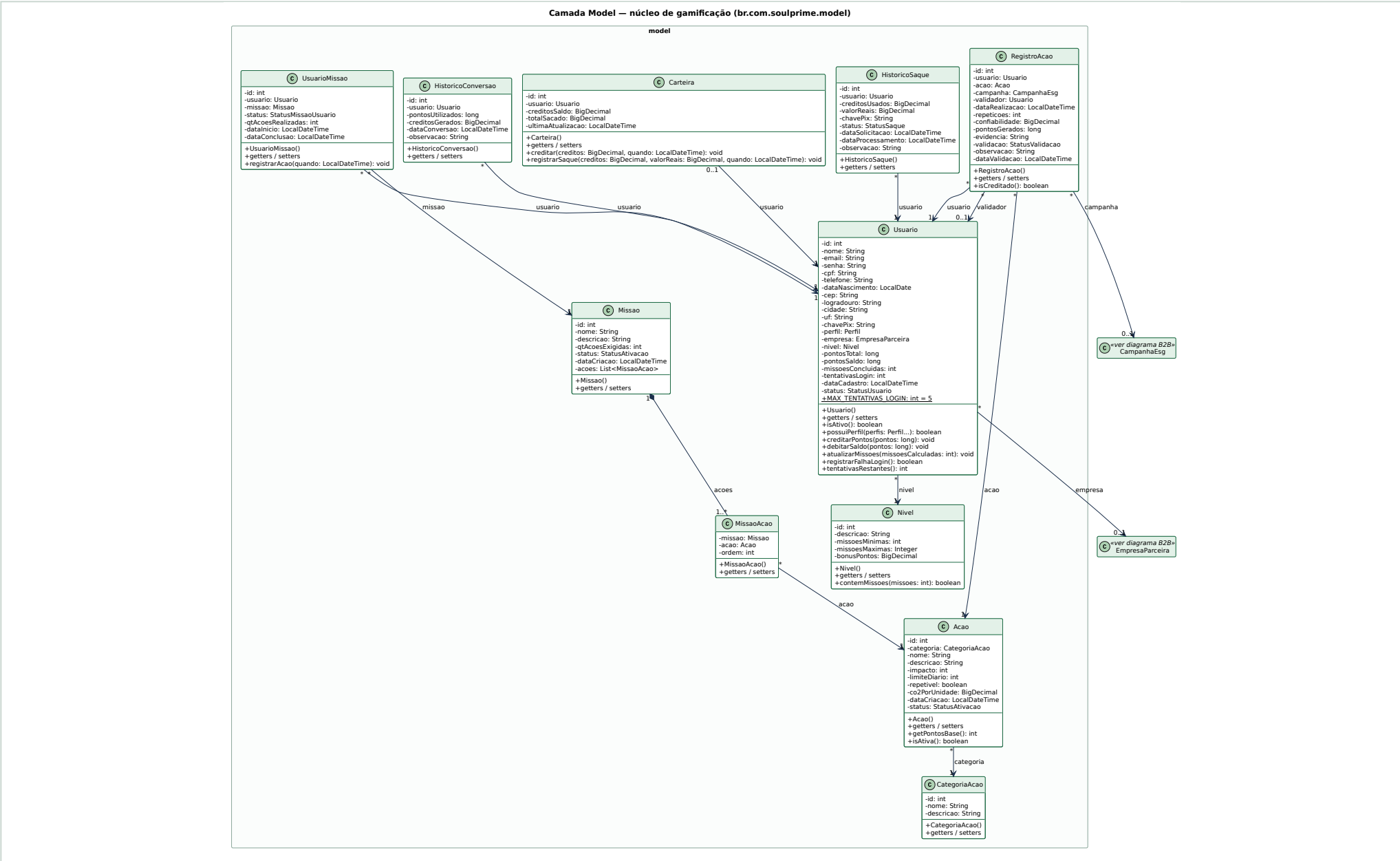


Figura 7.1 — Camada Model: núcleo de gamificação.

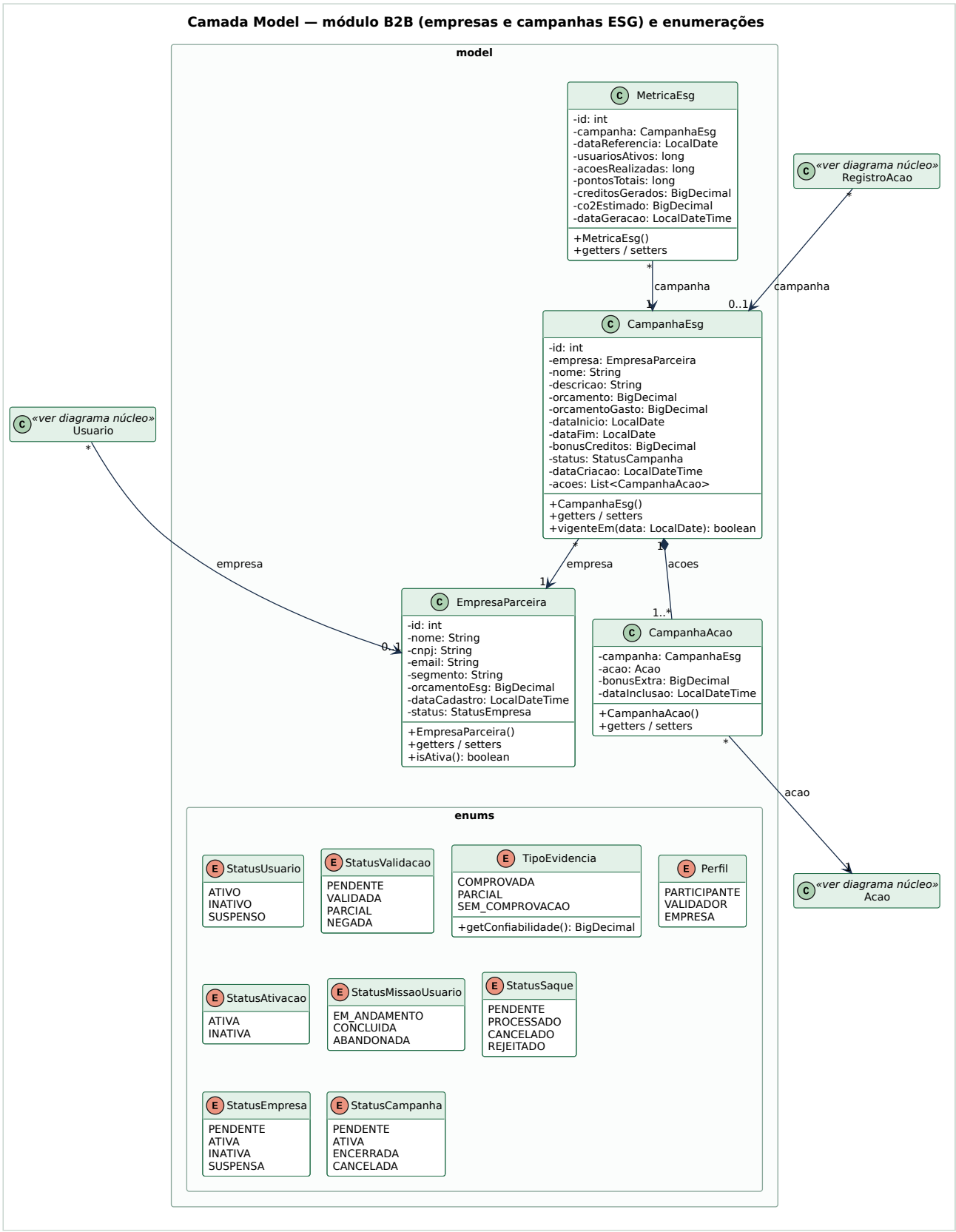


Figura 7.2 — Camada Model: módulo B2B e enumerações.

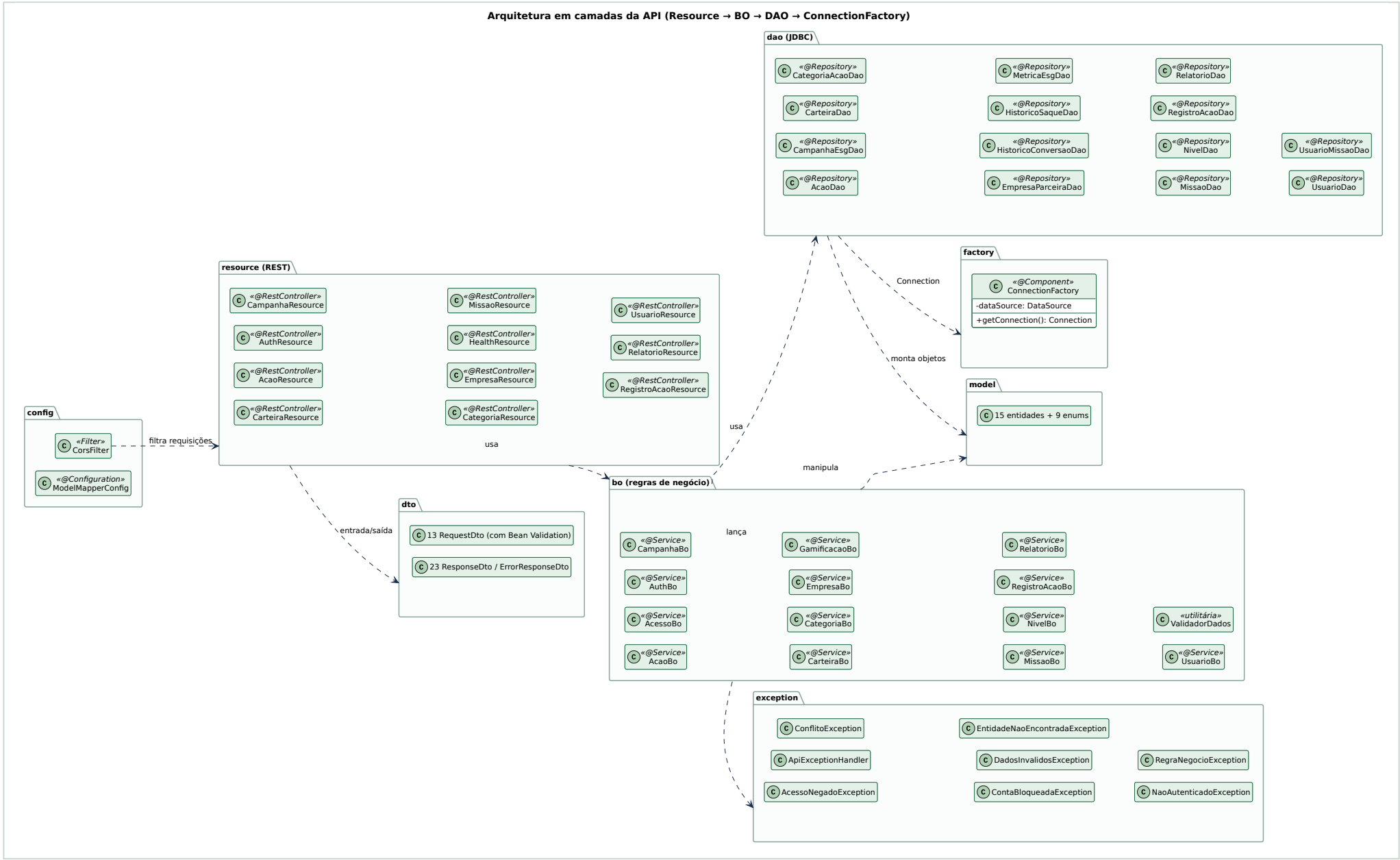


Figura 7.3 — Camadas da aplicação: Resource → BO → DAO → ConnectionFactory.

8. Procedimentos para rodar a aplicação

8.1 Ferramentas e versões

Ferramenta	Versão	Uso
Java JDK	21 ou superior	Compilar e executar
Maven	Maven Wrapper incluso (`mvnw`); não precisa instalar	Dependências e build
IDE	IntelliJ IDEA, Eclipse ou VS Code (Extension Pack for Java)	Desenvolvimento
Oracle Database	19c/21c (oracle.fiap.com.br)	Banco de dados
Oracle SQL Developer	qualquer versão recente	Executar o DDL e a carga
Docker (opcional)	24+	Executar em contêiner e fazer o deploy

8.2 Banco de dados

No SQL Developer, execute com F5 os scripts da disciplina de Building Relational Database:

1. `00_EcoScore_DDL.sql`;
2. `01_EcoScore_Carga_Inicial.sql`.

Todos os usuários de teste usam a senha `Eco@2026` (ex.: participante `ana.lima@email.com`, validador `patricia.andrade@gaiatech.com.br`, empresa `marcos.duarte@ecoverde.com.br`).

8.3 Variáveis de ambiente

Copie `.env.example` para `.env` (o arquivo não vai para o GitHub) e preencha:

```
DB_URL=jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl
DB_USER=rmXXXXXX
DB_PASSWORD=********
PORT=8080            (opcional)
CORS_ORIGINS=*       (opcional; URL do Front em produção)
```

8.4 Executar

Terminal (Linux/macOS):

```
cd EcoScoreEngine-API
set -a; . ./.env; set +a
./mvnw spring-boot:run
```

Windows (PowerShell):

```
$env:DB_URL="jdbc:oracle:thin:@oracle.fiap.com.br:1521:orcl"
$env:DB_USER="rmXXXXXX"
$env:DB_PASSWORD="********"
.\mvnw.cmd spring-boot:run
```

Importar na IDE:

- **IntelliJ:** *File → Open* → selecione o `pom.xml` → *Open as Project*;
- **Eclipse:** *File → Import → Maven → Existing Maven Projects*.

Depois de importar, configure as 3 variáveis de ambiente na *Run Configuration* e execute a classe `EcoScoreEngineApiApplication`.

Docker:

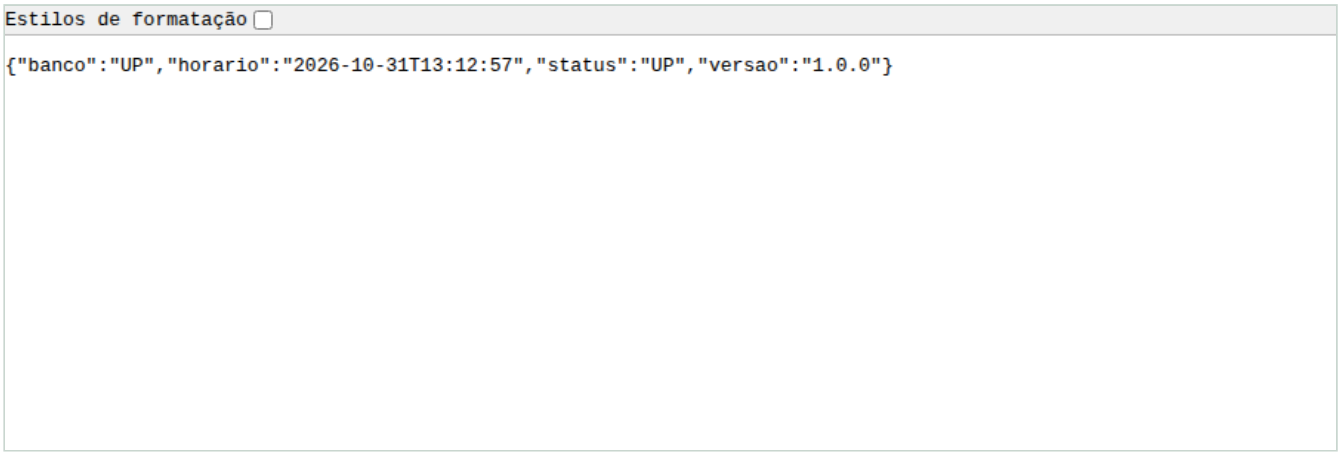
```
docker build -t ecoscore-api .
docker run -p 8080:8080 --env-file .env ecoscore-api
```

Verificar: abra `http://localhost:8080/health`. A resposta esperada é `{"status":"UP","banco":"UP",...}`. Os endpoints podem ser testados com Postman, Insomnia ou curl.

9. Deploy na nuvem

A API é publicada no **Render** (Web Service com Docker, plano gratuito). O `Dockerfile` do projeto compila com o Maven Wrapper e executa o jar em uma imagem só com o JRE 21. A porta vem da variável `PORT`, e as credenciais do banco são configuradas como variáveis de ambiente do serviço.

Item	Endereço
API (URL base)	https://ecoscoreengine-api.onrender.com
Saúde da API	https://ecoscoreengine-api.onrender.com/health
Repositório GitHub	https://github.com/Leonardo-2112/EcoScoreEngine-API
Front-end (Vercel)	**a preencher**



Resposta do endpoint /health na URL pública do Render (banco UP).

10. Testes realizados

A API foi testada de ponta a ponta contra o Oracle da FIAP com um roteiro automatizado de requisições HTTP: **151 de 151 casos aprovados**. Os testes cobrem todos os endpoints, os fluxos de sucesso e os de erro. O enunciado dispensa a classe de teste quando a API está completa; o roteiro foi usado para comprovar o funcionamento.

Grupo	Casos	Aprovados	Status HTTP verificados
Página inicial / rotas inválidas	2	**2**	200, 404
Saúde da API	1	**1**	200
Níveis	1	**1**	200
Categorias	10	**10**	200, 201, 204, 403, 404, 409
Catálogo de ações	12	**12**	200, 201, 204, 400, 404, 409
Desafios temáticos	7	**7**	200, 201, 204, 404, 422
Ranking	4	**4**	200, 405
Campanhas ESG	16	**16**	200, 201, 204, 400, 401, 403, 409, 422
Autenticação e cadastro	26	**26**	200, 201, 400, 401, 403, 409, 423
Usuários, painel e carteira	34	**34**	200, 201, 204, 400, 401, 403, 404, 409, 422
Registro e validação de ações	23	**23**	200, 201, 400, 403, 404, 422
Empresas (B2B)	12	**12**	200, 201, 400, 403, 409, 422
Dashboard ESG	3	**3**	200, 403

10.1 Casos de regra de negócio verificados

- Ação comprovada creditada na hora (20 pts); 2º registro do dia vale metade (7 pts) e fica PENDENTE; 3º bloqueado (422)
- Validador aprova (credita) ou nega (0 pontos, justificativa obrigatória); registro já decidido não pode ser revalidado (422)
- 3ª ação creditada conclui 1 missão; o desafio “Primeiros Passos” passa a CONCLUIDA
- Conversão de 293 pontos gera 2 créditos e o ranking não muda; saque de 12 créditos rende R$ 1,20 (PIX do cadastro); saldo insuficiente dá 422
- 5 senhas erradas bloqueiam a conta (423); o validador desbloqueia e o login volta a funcionar
- Empresa nasce PENDENTE e não faz login até ser ativada (RN03); campanha precisa de aprovação (RN16) e não pode ser editada depois de ATIVA
- Isolamento por perfil: participante não vê dados de outro (403); empresa não vê registros individuais (RN17)
- CPF/CNPJ com dígito verificador inválido, menor de 16 anos e JSON inválido dão 400; e-mail, CPF ou CNPJ duplicado dão 409